

SAGE-K8s — Final Week Plan (Demo-First)

Mon 5 Oct, 19:00 → submission. **Demo is Wednesday 7 Oct**. Phase 1 = demo sprint (tonight + Tuesday). Phase 2 = full submission sprint (Wed after demo → Sunday). Arya · Ananya · Subhashini

Project completion — roughly 85% · system works; results unarchived; no deck, no report, no recorded demo



TONIGHT, FIRST — check this within ten minutes.
Is the Azure VM still alive? Everything below forks on the answer.

- VM alive** → pull the audit PVC data off it immediately (Arya Task 1), leave it running until after Wednesday, and take a VM snapshot as insurance. Do not destroy it again this week.
- VM destroyed** → your Week 3 audit data is probably gone with its disk. Say so now, out loud, to the whole team. Tuesday morning becomes rebuild + redeploy + re-verify, and the experiment arms get re-run Tuesday afternoon (~3 hours of cluster time). The demo still works. The numbers are recoverable. Only the time is not.

Either way: **no VM teardown until the project is submitted**. The standing per-session teardown rule is suspended for the rest of the week — the cost of a few days of uptime is trivial against losing the data or the demo environment.

PHASE 1 — DEMO SPRINT · Tonight + Tuesday · Demo Wednesday

The one thing to understand about your demo: rollback firing is **fast** — Week 2 measured `rolled_back` at 20.0s and 30.0s. The 4m33s figure is the full three-attempt exhaust, not the rollback. So your contribution *is* demonstrable live. Show rollback firing in real time, then show `exhausted` from the archived audit log on screen instead of making anyone wait four and a half minutes.

Tonight, Monday (3 focused hours max — do not sprawl)

Who	Task	Done when
Arya	1. VM status check. If alive: <code>kubect1 cp</code> the audit PVC contents + all per-run dirs to local disk via a temporary pod that mounts it. Verify the copy (line counts, file sizes). Commit under <code>runs/</code> , push. Second copy to cloud storage.	Data exists in two places, neither of them that VM
Arya	2. Smoke-test the full loop once in deployed mode. Confirm W2 still goes detect → <code>rollout_undo</code> → recovered.	One green end-to-end run tonight
Subhashini	3. Answer two questions in writing to the team: (a) were the adversarial + legitimate labelled sets built and scored? (b) did the experiment runs use the live classifier or the mock? Then confirm Grafana comes up and the incident-lifecycle panel renders.	Both answered; Grafana reachable
Ananya	4. Review the <code>safety.AuditEntry</code> diff merged into your package without approval — approve or raise. Then state whether the UID-based pre-action pod rejection landed.	Both answered
All	5. 20-minute call: fill the inventory table (next page). Every cell gets a number or MISSING.	Table filled

The inventory table — fill this tonight

Artifact needed for the demo	Exists?	Owner
Audit data from Week 3 runs, archived off-VM		Arya
W1 / W2 / W3 manifests, runnable from the runbook		Arya
Grafana incident-lifecycle panel showing real data		Subhashini
Rollback rate (attempt-level + incident-level)		Subhashini
Recovery per workload per arm (enabled vs disabled)		Subhashini
TTM decomposition (detect / classify / apply / verify)		Ananya + Subhashini
Validator scores — adversarial AND legitimate		Subhashini
Classifier latency + cost per call		Subhashini

Rule: anything MISSING that is cheap gets rebuilt Tuesday morning (labelled sets need no cluster time at all). Anything MISSING that needs cluster time gets one slot Tuesday afternoon. Anything still missing by Tuesday 18:00 is **left out of Wednesday's results slide and stated as "in progress"** — never guessed.

Tuesday — build, rehearse, record (the whole day)

ARYA Demo mechanics and the fallback recording

T1 — Write the demo as a literal script, command by command, with expected output at each step and the exact timings below. Not notes — a script you can read under pressure.

Segment	Target time	What you run / show	What it proves
Context	0:00–0:40	One architecture slide. One sentence: "detects CrashLoopBackOff, fixes it from a two-action reversible allowlist, verifies the fix held for 60s, rolls back if it didn't."	Framing
Happy path	0:40–2:00	Inject W2 (bad revision). Live: detect → classify → rollout_undo → verified recovered (~65s). Grafana panel + controller logs side by side.	Autonomous remediation works, no human
Rollback (the contribution)	2:00–3:00	Inject W3 (both revisions bad). Live: fix applied → verification fails → rollback fires in ~20–30s, snapshot restored.	The safety layer actually fires — this is the project
Knowing when to stop	3:00–3:40	Show the archived audit log on screen: 3 attempts, then exhausted , then silence. Say plainly: "that takes 4m33s end to end, so here it is from a recorded run."	No infinite remediation loop
Why we measure a baseline	3:40–4:30	Inject W1 (transient) with the controller disabled. Kubernetes recovers it unaided.	Sometimes K8s would have fixed it anyway — hence the null control
Numbers	4:30–6:00	Results table + the comparison table against Wiesinger and ARBITER.	We measured it

Also prepare a 2-minute cut (context → W2 → W3 rollback → results) in case they give you less time than you expect. Know which segments you drop and in what order.

T2 — Record every segment on video. This is the insurance and it is not optional. Record each segment separately so you can play just the one that fails. Keep the recordings on a laptop you bring, not only in the cloud.

Assume the live demo breaks. Clusters misbehave in front of audiences — that is the oldest law in this field. A team that says "the cluster is being slow, here's the recorded run while it catches up" looks prepared. A team that stares at a hanging terminal does not. Record Tuesday.

T3 — Rehearse end to end three times. Timed, out loud, with someone interrupting. The third run is the one that finds the real problems.

T4 — Pre-flight checklist for Wednesday morning: VM up, K3s healthy, all pods ready, controller deployed at the frozen image, Grafana loaded with the panel open, three workload manifests staged in separate terminals, archived log open in an editor, recordings open in a player, deck open, laptop charged, screen mirroring tested.

SUBHASHINI Numbers on a slide, and the gaps closed

T5 — Rebuild whatever the inventory marked MISSING, in this order: labelled sets first (no cluster time, pure desk work), then live-classifier runs, then any missing arm.

T6 — Write results.py that regenerates the whole results table from runs/. Not a hand-typed table — a committed script, so every number is reproducible and cannot drift between the slide, the report and the repo.

Paste to your AI: "I have per-run directories under runs/, each with audit.jsonl and meta.json (workload, arm, timestamps). Write a Python script that regenerates my results table: rollback rate over attempts and over incidents; recovery rate per workload per arm; attributable recovery as enabled minus disabled; TTD and TTM decomposed into detect/classify/apply/verify; distribution across five terminal outcomes; and validator rejection and acceptance rates from a separate labelled-set file. Output markdown and JSON. Fail loudly with a clear message on any incomplete or malformed run rather than silently skipping it."

T7 — Two figures for Wednesday (the rest can wait for the report): outcome distribution per arm as a stacked bar, and TTM decomposition as a stacked bar that makes the 60s verification floor visible.

T8 — Grafana: export the dashboard as JSON and commit it. If the VM is ever rebuilt, the panel comes back in one import instead of an afternoon.

ANANYA The deck, and the hard questions

T9 — Build a 10-slide demo deck (the full 15-slide review deck comes in Phase 2):

1 Title · 2 Problem in one sentence · 3 The gap (what existing tools do and don't) · 4 Architecture diagram · 5 The loop (detect-classify-validate-snapshot-act-verify-rollback) · 6 The safety layer in detail – this is your slide · 7 Experimental setup (3 workloads, 6 arms, N=5) · 8 Results · 9 Comparison vs Wiesinger / ARBITER · 10 Limitations + future work

Slide 10 is not optional and not a weakness. Naming your own limitations before anyone asks — the 60s TTM floor, untuned window constants, N=5, single cluster — is the clearest signal of research maturity you can send. Examiners who find a limitation you already listed stop hunting for ones you missed.

T10 — Draw the architecture and sequence diagrams (needed Wednesday, reused in the report).

Paste to your AI: "Give me Mermaid source for two diagrams of a Kubernetes self-healing operator. (1) A component architecture diagram: controller watching pod events, an LLM classifier, a deterministic validator with an action allowlist, a safety layer doing snapshot/verify/rollback, the K3s cluster, and Prometheus/Grafana. (2) A sequence diagram for one incident: detection → evidence capture → classify → validate → snapshot → remediate → 60s verification window → either recovered or rollback → audit write. Keep both readable when printed in greyscale on A4."

ALL THREE Tuesday evening — 45-minute Q&A drill

- One answers, two attack. Everyone must be able to answer everything; examiners choose, you don't.
- "Doesn't Kubernetes already restart crashing pods?" → Yes, and that is exactly why we ran a controller-disabled control arm. Here is the measured baseline per workload and recovery net of it.
 - "What's new versus ARBITER?" → They built the safety layer; we measured it. Their rollback monitor covers two action types and no reported experiment shows it firing; their 5/5 is action selection, not the safety mechanism; every ARBITER experiment runs approval-required by their own account. We run ungated on one fault class and report the rollback rate.
 - "Why 60 seconds?" → Chosen a priori, not tuned. Stated as a limitation. No paper in our surveyed corpus evaluates window sensitivity either.
 - "Why only two actions?" → Reversibility. Every action must be undoable by restoring a snapshot — that is what makes unsupervised execution defensible at all.
 - "What if the LLM is wrong?" → It cannot act. It proposes; a deterministic Go validator checks the action against the allowlist, the target against live cluster state, and a semantic guard against the evidence. Off-allowlist output is rejected and escalated.
 - "Which parts did each of you write?" → Specifically, including what you did not write.
 - "Show me it failing." → That is segment three. Have it ready rather than treating failure as the awkward case.

Wednesday — demo day

Run the pre-flight checklist 45 minutes before, not 5. Arya drives the terminal, Ananya narrates the architecture and safety layer, Subhashini takes the results and comparison questions. If something hangs, say what you expected to happen and switch to the recording without apologising for the cluster.

PHASE 2 — SUBMISSION SPRINT · Wednesday after the demo → Sunday

The hidden deadline that drives this phase. The **plagiarism/similarity check** has a turnaround you do not control, and most colleges require the report to clear it before submission. That means the report must be **substantially complete by Thursday evening**, not Sunday. Work backwards from that. Find out tomorrow which tool your department uses and what the turnaround is — this single fact can wreck a submission and it is a five-minute question to your guide.

Day-by-day

Day	Arya	Ananya	Subhashini
Wed (post-demo)	Code freeze, tag v1.0-viva . Repo README with architecture diagram + results summary.	Ch 4 System Design + all diagrams.	Fix the three review-paper errors. Final results table.
Thu	Ch 5 Implementation + Ch 6 Testing. Clean-machine reproducibility test.	Ch 3 Requirements + Threats to Validity .	Ch 2 Literature Survey (from the review paper) + Ch 7 Results.
Thu eve	Assemble the full report and submit it to the plagiarism check. Front matter can still be rough; the body cannot.		
Fri	Appendices (manifests, key code). Runbook final.	Ch 1 Introduction + Ch 8 Conclusion & Future Work.	Front matter, references in IEEE format, figures finalised.
Sat	Full 15-slide review deck. Guide review and sign-off. Act on plagiarism results. Full viva drill.		
Sun	Formatting to the college template, certificate and declaration pages, final proofread, submit. Update resumes with real numbers.		

Complete deliverables checklist

A · Code and data

- runs/ committed – every audit log + meta.json
- Second backup off-VM
- runs/README.md mapping run → arm, workload, timestamp, outcome
- Labelled sets (adversarial + legitimate) committed
- results.py regenerates the full table from runs/
- Grafana dashboard exported as JSON and committed
- Code frozen and tagged v1.0-viva
- All tests pass, CI green
- Repo README: what it is, architecture diagram, quickstart, results summary
- Runbook validated by someone other than its author on a clean machine

B · Report

- Front matter: cover, certificate, declaration, acknowledgement, abstract, ToC, list of figures, list of tables, abbreviations
- Ch 1 Introduction – problem, objectives, scope, report organisation
- Ch 2 Literature Survey – adapted from your review paper
- Ch 3 Requirements – hardware, software, functional, non-functional
- Ch 4 System Design – architecture, module design, **5 diagrams**: architecture, control loop, state machine, sequence, deployment
- Ch 5 Implementation – operator, classifier, validator, safety layer, attempt budget, evidence freezing
- Ch 6 Testing – unit and integration tests, test-case table, the Week 2 bug findings
- Ch 7 Results – four contributions, 3 figures, comparison table
- Ch 8 Conclusion and Future Work
- References, IEEE format
- Appendices – workload manifests, key code listings
- Individual contribution statement per member
- Plagiarism report attached
- Guide sign-off

C · Presentation and demo

- 15-slide review deck (extends Tuesday's 10)
- Deck exported as PDF as a backup
- Demo script, written
- Recorded demo video – every segment
- 2-minute cut for interviews

D · Viva readiness

- Q&A bank rehearsed by all three
- Each owner can whiteboard their component cold, no notes
- Each owner has a 90-second interview narrative for their component
- Agreed one-sentence answers to: what's new, why 60s, why two actions, doesn't K8s do this already

E · Career artifacts

- Resume bullets updated with real measured numbers, past tense
- Repo public and presentable – interviewers will open it
- Review paper corrected (three fixes) and filed

If you fall behind, cut in this order. Do not let a lower item eat a higher one.

1. Keep: data archived, demo working, report body, plagiarism clearance, guide sign-off.
2. Trim first: the third results figure; appendix code listings; the 2-minute interview cut.
3. Trim next: review-paper corrections (do them, but they can slip past submission — just never submit both documents with the contradiction live).
4. Never cut: Threats to Validity, the comparison table, the recorded demo, anything that makes a number traceable.

Four rules for the rest of the week.

- **No new features.** The system is done. Every hour spent adding capability is an hour not spent being able to explain it.
- **No number anywhere — slide, report, resume — that results.py cannot regenerate from an archived run.**
- **No VM teardown until submission.**
- **No claim of priority over ARBITER**, and never say either comparator "has no recovery mechanism" — Wiesinger retries iteratively, ARBITER has a monitor. State precisely what is missing instead.